Algorithms pseudo-code - Verifier - Block 3

Client	Post CH AG
Date	April 23, 2020
Version	0.3
Status	Draft
Classification	Confidential
Author	Thomas Hofer
Distribution	Olivier Esseiva, Nils Aellen, Post CH AG
Modifications	

Route Cité-Ouest 19, CH-1196 Gland
+41 22 364 85 70
info@objectif-securite.ch

OS OBJECTIF SÉCURITÉ
Architecte de la sécurité informatique

Contents

Introduction

This document is part of Swiss Post's sVote solution, based on previous work by Scytl.

Aim of the document

This document aims to provide all the information and pseudo-code needed to implement the so-called "block 3" of an independent verifier for the Swiss Post's internet voting solution. This covers the verification of all cryptographic proofs generated over the course of an election event, such as:

- proofs of the validity of the votes cast, generated by the voters' voting platform,
- proofs of the validity of the mixing (i.e. all the votes received by the control components have been shuffled),
- proofs of the validity of the partial decryptions (i.e. all the votes shuffled have been decrypted using the previously commited key).

See also

TODO: list of other relevant documents

Version information

This document was built on commit 2936365, dated Thu Apr 23 13:04:58 2020 +0200.

Changes since last document delivery

Hash	Date	Comment
2936365	2020-04-23 13:04	Fix typos in RandomOracleHash
bdb2981	2020-04-23 12:59	Update test data files for arguments, after changes to RandomOracleHash
8a6ba16	2020-04-23 11:40	Rewrite RandomOracleHash

Breaking Changes

This section aims to keep track of the modifications on previously defined and implemented algorithms, which will need to be adapted. Added elements are not tracked here, to keep it as concise as possible. To the extent possible, such changes are marked in red in the text.

Version 0.3

- Algorithm 1.2.2 has been rewritten to conform to the updated eVoting specification.

Version 0.2

- Algorithm 1.5 can now accept a vector $\vec{a}$ shorter than the commitment key.

1 Utility functions

1.1 Conversion algorithms

1.1.1 Integers to Strings

When they need to be represented as strings, numeric values are written as their decimal representation.

1.1.2 Strings to Bytes

When strings need to be represented as arrays of bytes, their UTF-8 encoding is used.

1.1.3 String representation of objects

In the current implementation of the voting system, whenever elements are passed to a hash function, their `toString` method is used to convert them to strings. This implies that the representation of objects is highly dependent on the implementation itself.

Ideally, in the longer run, the representation of objects will be made easier and less coupled to the original implementation.

1.1.3.1 Exponent

Inputs:	v the value of the exponent
	q the cardinality of the subgroup
Output:	`"Exponent [q="`$integerToString(q)$`", value="`$integerToString(v)$`"]"`
Suggested name:	`exponentToString`

1.1.3.2 ZpGroupElement

Inputs:	v the value of the element
	p the congruence class
	q the cardinality of the subgroup
Output:	`"ZpGroupElement [value="`$integerToString(v)$`", "` `"p="`$integerToString(p)$`", q="`$integerToString(q)$`"]"`
Suggested name:	`zpGroupElementToString`

1.1.3.3 PublicCommitment

Inputs:	v the value being committed
	p the congruence class
	q the cardinality of the subgroup
Output:	`PublicCommitment [_commitment="` $zpGroupElementToString(v, p, q)$ `"]"`
Suggested name:	`publicCommitmentToString`

1.1.3.4 SingleValueProductProofInitialMessage

Inputs:	c_d the first value being committed
	c_δ the second value being committed
	c_Δ the third value being committed
	p the congruence class
	q the cardinality of the subgroup
Output:	`"SingleValueProductProofInitialMessage "` `"[cd="` $publicCommitmentToString(c_d, p, q)$ `", "` `"commitmentPublicLowDelta="` $publicCommitmentToString(c_\delta, p, q)$ `", "` `"commitmentPublicHighDelta="` $publicCommitmentToString(c_\Delta, p, q)$ `"]"`
Suggested name:	`singleValueProductProofInitialMessageToString`

1.1.3.5 CommitmentParams

Inputs:	h an element
	$(g_1, ..., g_n)$ a vector of elements
	p the congruence class
	q the cardinality of the subgroup
Output:	`"CommitmentParams [h="` $zpGroupElementToString(h, p, q)$ `", g=["` $\forall_{i=1}^{n-1} zpGroupElementToString(g_i, p, q)$ `", "` $zpGroupElementToString(g_n, p, q)$ `"]"`
Suggested name:	`commitmentParamsToString`

1.1.3.6 ZeroProofInitialMessage

Inputs:	c_{a_0} a commitment to a value
	c_{b_m} a commitment to a value
	$c_D = (c_{D_1}, ..., c_{D_n})$ a vector of commitments
	p the congruence class
	q the cardinality of the subgroup
Output:	`"ZeroProofInitialMessage "` `"[_cA0="` $publicCommitmentToString(c_{a_0}, p, q)$ `", _cBM="` $publicCommitmentToString(c_{b_m}, p, q)$ `", _cD=["` $\forall_{i=1}^{n-1} publicCommitmentToString(c_{D_i}, p, q)$`", "` $publicCommitmentToString(c_{D_n}, p, q)$ `"]"`
Suggested name:	`zeroProofInitialMessageToString`

1.1.3.7 HadamardProductProofInitialMessage

Inputs:	$c_B = (c_{B_1}, ..., c_{B_n})$ a vector of commitments
	p the congruence class
	q the cardinality of the subgroup
Output:	`"HadamardProductProofInitialMessage "` `"[_cB="` $\forall_{i=1}^{n-1} publicCommitmentToString(c_{B_i}, p, q)$`", "` $publicCommitmentToString(c_{B_n}, p, q)$ `"]"`
Suggested name:	`hadamardProductProofInitialMessageToString`

1.2 Hashing algorithms

Several elements below make use of the Fiat-Shamir heuristic to turn interactive cryptographic proofs into non-interactive ones. To that end, the prover and the verifier must agree on how to hash the required data so that the non-interactive challenges will result in the same value for both parties.

This section describes which hashing primitive is used and how the hashes of multiple input values are combined. As such, this documents acts as a formalisation of the method used in Swiss Post-Scytl's sVote internet voting system's, so that a compatible verifier can be implemented.

1.2.1 Hashing primitive

The hashing primitive used throughout the system is SHA2-256, for which reference implementations exist in common programming languages. Other parts of the solution may reference SHAKE-128, but as far as the cryptographic proofs are concerned, only SHA2-256 is used.

1.2.2 Random Oracle Hash

This is a complete rewrite of the function, to match the reviewed and fixed version of April 2020.

This impacts the test values provided for all algorithms relying on this one.

This primitive is used to map an input value $\in [0,1]^n$ to a *random-looking* value $\in \mathbb{Z}_q$. This is used by the proofs relying on the Fiat-Shamir heuristic to turn an interactive protocol into a non-interactive proof system.

Inputs:	A hash function H
	An input value $x \in [0,1]^n$
	A lowerbound $lowerBound \in \mathbb{Z}$, typically 0
	An upperbound $upperBound \in \mathbb{Z}$, typically q
Output:	A *hash* of the input value, $lowerBound \leq output < upperBound$.
Suggested name :	`randomOracleHash`

Require: H a cryptographically secure hash-function, with output size s (in bytes)
Require: $lowerBound < upperBound$
$c \leftarrow upperBound - lowerBound - 1$
$t \leftarrow |c|$ (i.e. the bitLength of c)
$l \leftarrow \lceil t/8 \rceil$
$input \leftarrow x$
if $l \leq s$ **then**
  $e \leftarrow$ the l first bytes of $H(input)$
else
  $q \leftarrow \lfloor l/s \rfloor$
  $r \leftarrow l \bmod s$
  $e_0 \leftarrow H(input)$
  for $i \in [1, q-1]$ **do**
    $e_i \leftarrow H(e_{i-1}|" : "|input)$
  end for
  $e \leftarrow e_0|...|e_{q-1}$
  if $r > 0$ **then**
    $e_q \leftarrow Trunc_r(H(e_{q-1}|" : "|input))$ {i.e. keep only the r first bytes of the hash}
    $e \leftarrow e|e_q$

20 **end if**
21 **end if**
22 {Now the first byte of e needs to be masked so that e only contains the required number of
23 bits...}
24 $e[0] \leftarrow e[0] \,\&\, \&(255 \gg (8 - t\,\%\,8))$ {That is we only keep the $t\,\%\,8$ least significant bits of the first
25 byte. **Note that when $t\,\%\,8 = 0$, the resulting value is 8 bits shorter than expected.**}
26 $y' \leftarrow$the unsigned integer value whose magnitude is given by e
27 **if** $y' \geq c$ **then**
28 $input \leftarrow e$
29 Go back to line 7.
30 **end if**
31 $y \leftarrow (y' + lowerBound + 1) \bmod upperBound$

Examples Some examples are provided in the attached file .

1.2.3 Inverted order string concatenation

Similarly to the warning in paragraph 1.2.2, this differs from the specification and is only required temporarily to ensure compatibility with the non-compliant implementation.

Input:	A vector of strings $(s_1, ..., s_n)$
Output:	A single concatenated string, with reverse order of elements, *i.e.* $s_n \vert s_{n-1} \vert ... \vert s_1$
Suggested name:	`reverseAndJoin`

1.3 Data-model to primitives

1.4 BigInteger math abstraction layer

1.4.1 Modular exponentiation

Inputs:	b the base: `BigInteger`
	e the exponent: `BigInteger`
	m the modulus: `BigInteger`
Output:	the modular exponentiation $b^e \bmod m$
Notes:	the naive implementation (before optimisation) can simply consist of calling `b.modPow(e,m)`.
Suggested name:	`modExp`

Simple examples

b	e	m	output
2	3	5	3
3	5	11	1
5	3	11	4

Real-size values

b	0x4
e	0x123456789ABCDEF0123456789ABCDEF0123456789ABCDEF0123456789ABCDEF0123456789ABCDEF0123456789ABCDEF0123456789ABCDEF0123456789ABCDEF0123456789ABCDEF0123456789ABCDEF0123456789ABCDEF0123456789ABCDEF0123456789ABCDEF0123456789ABCDEF0
m	0xB7E151628AED2A6ABF7158809CF4F3C762E7160F38B4DA56A784D9045190CFEF324E7738926CFBE5F4BF8D8D8C31D763DA06C80ABB1185EB4F7C7B5757F5958490CFD47D7C19BB42158D9554F7B46BCED55C4D79FD5F24D6613C31C3839A2DDF8A9A276BCFBFA1C877C56284DAB79CD4C2B3293D20E9E5EAF02AC60ACC93ED874422A52ECB238FEEE5AB6ADD835FD1A0753D0A8F78E537D2B95BB79D8DCAEC642C1E9F23B829B5C2780BF38737DF8BB300D01334A0D0BD8645CBFA73A6160FFE393C48CBBBCA060F0FF8EC6D31BEB5CCEED7F2F0BB088017163BC60DF45A0ECB1BCD289B06CBBFEA21AD08E1847F3F7378D56CED94640D6EF0D3D37BE69D0063
output	0x8DE3DB1E8E54041EA83E88C31A8FE20BBA09162BE288F5D795FA0596309CCB597A969DEDEDC70C8B305A67C2CE8839A315FA368FF90800F7E2A3913A522B820ACF1F5E0F0020F1CA5E539AA082AE7ABD46B6B0354B925A2A9C9D8694050042C0DA2BAFBE122B015C35091794CB70EFA3C35EFB4C6985A7D97F24BAC4F3A068AC18A11B80792A39434095D1BD32B7912586E66F28C011CBF15EF067F10B41C41549E80CF1A5985A643D95DD0AF87A7F014BCD880B1D07EF332231CC71AFE5242B6895A7DA331D3163AC635A42CE5795D936AB4A05431BCFD41582360E2DDE114D2A974016572A6E0D36F19247EB560077EAC068E4BB1B9CCD85A5B8F2389148F8

1.4.2 Modular exponentiations product

Inputs:	$\vec{b}$ a list of bases: `BigInteger`
	$\vec{e}$ a list of exponents: `BigInteger`
	m the modulus: `BigInteger`
Output:	the product of the modular exponentiations $\prod_{i=1}^{n} b_i^{e_i} \bmod m$
Notes:	the naive implementation (before optimisation) can simply consist of calling the modular exponentiation defined in paragraph 1.4.1 for each i, computing the product and taking the $\bmod p$.
Suggested name:	`modExpProduct`

Simple examples

b	e	m	output
(2, 3)	(5, 7)	11	2
(5, 7)	(3, 2)	11	9
(2, 3, 5)	(5, 2, 3)	7	6

Real-size examples

$\vec{b}$	(0x4, 0x9)
$\vec{e}$	(0x123456789ABCDEF0123456789ABCDEF0123456789ABCDEF0123456789ABCDEF01234567 89ABCDEF0123456789ABCDEF0123456789ABCDEF0123456789ABCDEF0123456789ABCDEF01 23456789ABCDEF0123456789ABCDEF0123456789ABCDEF0123456789ABCDEF0123456789AB CDEF0, 0xFEDCBA9876543210FEDCBA9876543210FEDCBA9876543210FEDCBA9876543210FEDCBA98 76543210FEDCBA9876543210FEDCBA9876543210FEDCBA9876543210FEDCBA9876543210FE DCBA9876543210FEDCBA9876543210FEDCBA9876543210FEDCBA9876543210FEDCBA987654 3210)
m	0xB7E151628AED2A6ABF7158809CF4F3C762E7160F38B4DA56A784D9045190CFEF324E77389 26CFBE5F4BF8D8D8C31D763DA06C80ABB1185EB4F7C7B5757F5958490CFD47D7C19BB42158D 9554F7B46BCED55C4D79FD5F24D6613C31C3839A2DDF8A9A276BCFBFA1C877C56284DAB79CD 4C2B3293D20E9E5EAF02AC60ACC93ED874422A52ECB238FEEE5AB6ADD835FD1A0753D0A8F78 E537D2B95BB79D8DCAEC642C1E9F23B829B5C2780BF38737DF8BB300D01334A0D0BD8645CBF A73A6160FFE393C48CBBBCA060F0FF8EC6D31BEB5CCEED7F2F0BB088017163BC60DF45A0ECB 1BCD289B06CBBFEA21AD08E1847F3F7378D56CED94640D6EF0D3D37BE69D0063
output	0x34D00F944346E0356AF5D6E912FF78CDA179AC699F64A806AA50979113FE25B32C4114EBA F34A752527B497BD734750FCCC778A59F15A82EF8F064709351000A2D0ED57E69FE49C529F5 1B7148FCA1FAEDA7F5AF4FA3AF21BFA58691A454B14AD9F4C07751CC1689EBF61433330D9D7 69C0549FF20EC25D1C366D302DC06B21FB7AA8C408E03A96DD1B3DBBFFE204BAC4319EFFD62 F6A0C2B32C233F9B4F8155203A552A23B79577CAFCE8642409FD2E6C2206E74B823FE6760BE F70825EF8B8EECC250D94D2370D23D5CACE39CBFA83E6D7A12DC50758D6303AD2221D850C2A BA9F21B97AE0A796021D9F9077DEE09C8E490B8709248B6CFBDB5F107E3C4021

1.4.3 Modular inverse

Inputs:	b the base: `BigInteger`
	m the modulus: `BigInteger`
Output:	the modular inverse $b^{-1} \bmod m$
Notes:	the naive implementation (before optimisation) can simply consist of calling `b.modInverse(m)`.
Suggested name:	`modInv`

Simple examples

b	m	output
2	5	3
3	11	4
5	11	9

Real-size examples

b	0x123456789ABCDEF0123456789ABCDEF0123456789ABCDEF0123456789ABCDEF0123456 789ABCDEF0123456789ABCDEF0123456789ABCDEF0123456789ABCDEF0123456789ABCDE F0123456789ABCDEF0123456789ABCDEF0123456789ABCDEF0123456789ABCDEF0123456 789ABCDEF0

m	0xB7E151628AED2A6ABF7158809CF4F3C762E7160F38B4DA56A784D9045190CFEF324E77 38926CFBE5F4BF8D8D8C31D763DA06C80ABB1185EB4F7C7B5757F5958490CFD47D7C19BB 42158D9554F7B46BCED55C4D79FD5F24D6613C31C3839A2DDF8A9A276BCFBFA1C877C562 84DAB79CD4C2B3293D20E9E5EAF02AC60ACC93ED874422A52ECB238FEEE5AB6ADD835FD1 A0753D0A8F78E537D2B95BB79D8DCAEC642C1E9F23B829B5C2780BF38737DF8BB300D013 34A0D0BD8645CBFA73A6160FFE393C48CBBBCA060F0FF8EC6D31BEB5CCEED7F2F0BB0880 17163BC60DF45A0ECB1BCD289B06CBBFEA21AD08E1847F3F7378D56CED94640D6EF0D3D3 7BE69D0063
output	0xB73CD0B3660FAF50A9F1F2614EFE4FD0A8A0F9258E15CD8BE119FFA1EA7ECBE1854A9E 45F0272A668285485753578B7803C4C8FC632F843D250008C43CE8AE721045043269EC25 4EAF3387356984EC62B6A7860BF1A483FB6853D1624987C2BA5483407D76B621677B5217 8DDC6DDF15ABE9E849072CC3AA320BBD40BA5329B441E2610B0BE802F31D48EA49E82E42 242BC7C2FE9D8B93FAEA1B249DCC8F6EEA10B8863AFF3E25E41B8580C627BE43C18B6CA8 14FDD5911456892299D7DAD554AC559269F2C9A7E5D452DC1B9AF878F5068020833797B9 0C52F9662F098A05DC558D6984AB1DAFA4419A09AA438E9F7D04AF58ABB24D51C84E8312 1AA2F25D05

1.4.4 Exponentiations product

This utility function computes the product of the elements of a vector raised to increasing powers of x.

Context:	Encryption group defined by (p, q)
Inputs:	$\vec{a} = (a_0, a_1, ..., a_{n-1}) \in \mathbb{G}_q^n$
	$x \in \mathbb{Z}_q$
Output:	$\prod_{i=0}^{n-1} a_i^{x^i}$
Suggested name:	`prodIncPow`

Pseudo-code

$result \leftarrow 1$
for $i \in [0, n-1]$ **do**
  $result \leftarrow result * modExp(a_i, modExp(x, i, q), p) \bmod p$
end for

Sample values Some samples values can be found in the attached file here: .

1.5 Commitment computation

Context:	Encryption Group (p, q, g)
Inputs:	Random exponent $r \in \mathbb{Z}_q$
	List of elements to be committed $\vec{a} = (a_1, ..., a_n) \in \mathbb{Z}_q^n$
	Commitment key $ck = (H, G_1, ..., G_m)$
Output:	The commitment value $com_{ck}(\mathbf{a}; r) \in \mathbb{G}_q$
Steps:	$com_{ck}(\vec{a}; r) = H^r \prod_{i=1}^{n} G_i^{a_i}$
Warning:	The $\mathrm{mod}p$ operator is implied... i.e. all exponentiations above are modular exponentiations and the products are to be computed $\mathrm{mod}p$ as well.
Suggested name:	`computeCommitment`

Commitment computation pseudo-code

Require: $r \in \mathbb{Z}_q$
Require: $\forall i \in [1, n] : a_i \in \mathbb{Z}_q$
Require: $H \in \mathbb{G}_q$
Require: $\forall i \in [1, m] : G_i \in \mathbb{G}_q$
Require: $n \leq m$
if $n < m$ **then**
  for $i \in [n + 1, m]$ **do**
    $a_i \leftarrow 0$
  end for
end if
$result \leftarrow modexp(H, r, p)$ {paragraph 1.4.1}
$result \leftarrow result \times modexpprod((G_1, ..., G_n), \vec{a}, p)$ {paragraph 1.4.2}
$result \leftarrow result \mathrm{mod}p$
Ensure: $result \in \mathbb{G}_q$
return $result$

Simple examples All elements in the table below assume $p = 11$ and $q = 5$

r	$\vec{a}$	H	$\vec{G}$	output
1	(2, 3)	4	(5, 9)	3
2	(1, 4)	9	(4, 3)	9

Real-size examples

p	0xB7E151628AED2A6ABF7158809CF4F3C762E7160F38B4DA56A784D9045190CFEF324E77389 26CFBE5F4BF8D8D8C31D763DA06C80ABB1185EB4F7C7B5757F5958490CFD47D7C19BB42158D 9554F7B46BCED55C4D79FD5F24D6613C31C3839A2DDF8A9A276BCFBFA1C877C56284DAB79CD 4C2B3293D20E9E5EAF02AC60ACC93ED874422A52ECB238FEEE5AB6ADD835FD1A0753D0A8F78 E537D2B95BB79D8DCAEC642C1E9F23B829B5C2780BF38737DF8BB300D01334A0D0BD8645CBF A73A6160FFE393C48CBBBCA060F0FF8EC6D31BEB5CCEED7F2F0BB088017163BC60DF45A0ECB 1BCD289B06CBBFEA21AD08E1847F3F7378D56CED94640D6EF0D3D37BE69D0063
q	0x5BF0A8B1457695355FB8AC404E7A79E3B1738B079C5A6D2B53C26C8228C867F799273B9C4 9367DF2FA5FC6C6C618EBB1ED0364055D88C2F5A7BE3DABABFACAC24867EA3EBE0CDDA10AC6 CAAA7BDA35E76AAE26BCFEAF926B309E18E1C1CD16EFC54D13B5E7DFD0E43BE2B1426D5BCE6 A6159949E9074F2F5781563056649F6C3A21152976591C7F772D5B56EC1AFE8D03A9E8547BC 729BE95CADDBCEC6E57632160F4F91DC14DAE13C05F9C39BEFC5D98068099A50685EC322E5F D39D30B07FF1C9E2465DDE5030787FC763698DF5AE6776BF9785D84400B8B1DE306FA2D0765 8DE6944D8365DFF510D68470C23F9FB9BC6AB676CA3206B77869E9BDF34E8031
r	0x1B7A50E7ED219F8F99EBD3B321E9C463E25A6EFF97F51DAE5F4DC09ED327A9343ADEB62A5 B877ED2D4A30F2B2E58FF7FAEE231C845AC56286D753C2195E1913FCAA859BFAC3C1B9DA27C 4501E667F5DA53ADC93711FCDA1D46282757AAF6DAD1C9CFE7786370C9D157F91FD53ADAF09 B9465B284403D4630503A99AFAC7B36CCC0BC24621868CCE18B9E7860CDEF1723C1D39E70CF 7F0B496DC97E4322CC545AFB5B16437EC6CF920B8EC8214CDD9899D60035BB81C473730AC4E F8A468E60ED99DE21B75489084370E5B7B41F2144892377C9B973262D4A0BD05D98B6328F60 01DAC978A62B7A868CC25849C66242451127D6CC0CC7AC09F85182B892FEAF76
a	(0x199A56BDE2D14B178711FBB69B69643A858CF27AE6CF16BA2906E906EC6323B0869CEFA 3C5B0260CD38CBDC4993D8B5ACB53675DDB0B5B1EBCAB62AA737CF5BB887C71EE8C938DA1A F19A5B7FB66DD1DF601E8FA696437062AE8ADF2040B7B8C2B39F3FC3BD60EF2EBF596698EF 290C8C07A68FD8E8DBCA94F2C44358B7C9077D32D941AA5AD4FC8D3E1B69CFDF65B14337B5 9F054F8C0F17467EFE9F0D7103C6F88E0806BA1575A94051776460AA01E419099C04B49C5E 7E743D457325928A1F2CA58897F83BFBBF682434F0BA05A9F65D841E47D71CD930410FDACF C9D2DF2B46D937822E31F42CAC2832362749F9F68068AE4D4AC6AC8507816EBE24BF0A1, 0x4FC2216FA9684363DEEAE833C54282DC3FD2FBA55615E951ECE94F0030DB27239636DF76 AC5D6769FD923978FBB693268C12C81C7F3A57798E1D4B0D11A1734A3CF530ECF3EA5B9F17 05B18FDF031B299B07067DA8FEB1ED9F436383A4BB2E512AEC00803C7F963EF96B6CDDD31F 84E1DF8095256EC43D49DCE2B2F70504D8C66C1BA2F1476305F7BB79D77C6E2242E3E1372A 33CBA0EF994957089E63429B0DC5AFCF19A15853D1E322FF7882F627E17E31949025FC14E7 9ED09A7E28EE2D14523ADBA5E29DB66D2BC75845FC9A45854B9BDDE7323A85392D81393044 EB7E23C2F3CA70C2CB2E9936193B61413F5019B8F8237F647E4311AF39FB16F1C87E1C, 0x452A304674FBEC25E2D9A8639335214C5BAA6659CCB2B1E8B28E7FE07FBF620B12A21F28 C1ED8536B649E1896B22ADA1FB36D26871140C34B26F71293A9539120B9F395F940F4B9071 07BE8A7C3D84D61FFC040236D246BC4FCC2FF8E03BFC2C41069EB6CAE3BBF9DBBC256CCFB1 E4FE8601109BD9BF233447BE2AAAC263026ACF91BA2294483A19FAD8253AAE1A04C4ABC9EB 92C8DA7D0BF06F2EB446895B482246A55BBD4A879CC0D235A345170807D37DFBBCD3B9D29C 5F2D896369F191B632417B850AECB0D46F96A700CDAC59CE58B8E8852F09F116B9264CDA41 06DA818CE1F4DB5A5D29B0165414A181E9A259342C6CB76FA69D4F322923690E5B7C83)
h	0x77BE550D2CC9F0BE0A6A45C578358248991949AE9FBC5DBC351DEB76475E5A3F8BAE24312 C0B918E10E89688F44BADB8EDF9DC1867EC5D86242560E2DB03DA09B6F1A72F394B6B590B09 B847922B537FBAFEEBAAA4D835D989880AD1A33A19C1E1D8ED64F40BFF12E310FD0B1BF7B53 B1FA9050A9EFADDA990CB3D5FF401B3717B8C313F9D64BE09CE96BF84D3BC694C77F13AFA5F DB1E4577CF1B5E926300EBE5CF05C0E33AECC2A7A72EAC87E0826004F39E716659E83BDB8EF 48EBB461F397A364BEAB3810C31D38720BBA3E669FC090B7107E1E6DBF6AD88236B4DC3DDD2 163EA1E853FD4AA954D1DDFD233DD7D71389E68AE4C64509C62BCF61F769E6EA

g	(0xC604C21B2AC4FF7A8B36BA0CC73945F3E89F6643DC3429B40D1BD1530D9DDC7BA5ADF90 F36870528AD72AACBAA8A5E7AAC28A0ADE36CB75A587C5DBC12F43E357E405947FD29902E5 2348444825704975981172 1F580FC96406B56E0DB6FD1B06541BFA6793C86271EFCCE7CA35 43306A3BA31258EC373D291C69278E3E23714EAC8A43C211BBB25EDB444A609EDE4492F10F BC7D1A9949305DD9B9987BD980265F501AEE840229F31478F19C6AB4AFE0B35B6801E859C8 3A3CF7CC7D3BFC9DF7D8DEB78FB0E0F8DBE64CEF3D97DAA0FC0A977F57968DF291BB80822D FCA62BED1CB893E7D90D485406093D8F3457F7407C7374CE52AA44E5D7594C6AD90B58, 0x16D328A54F725AC93184F49C7FC740FB4C48C39D2721FF29DBA190379C690DC9256BCFCA 2B83E3C01AAE10133907F108BBE693312E07B5F5F2C53D31BEBA3520785E2760D11140635C D89BC40AAD75FC349D5771D98675ACC8D7E7D8FCBAC30780D8F4A7AC6EDB4A005ADC0909E7 2013CDF63E764ABB116B259B225694BE92572BCA07723C44B4538CD025AD38E546D7D4DBAC 16D378AE1805D5C331CD4145AF3E580F313AF8139FC5652EE8CE078E97F334DA0820B110AF E424E5948CF7A99040D330F2B90C78A902DBC114E23A062B0801B6004F7CBF9D7DBB00C4A0 FA6B2F54B7D44A8066095F20B2636183C7BC3BC43B95595B1D134A48524946AD738F25, 0xA10D5EFD651D1383C2C6CE3A83AB5337DC4225AEA6BE8C796C822F6BB284D75EC2EC6747 F431310B3BFBCADEAF61C08246B3B0D7804AC01304FA318AEFD755203B72D3E1C87162F341 9202693FF044D0E20FDF86C90A3D774C05B314775128BA9F5926A0C663035940BA4E952805 66474A3FAAEDB8EDE0EEF70D26C6652271F4BEB790286144EAB2B4C17E7B2499B76643A2CB 32D8E71BE8A58CFE86522EB38E4EF5D92335689DF01ECC20FDE0B14C9E7781A3AE43DA82BD B7F30548C4251B3B4A1A74D9D09F65F86DFBD26F5B57339EAE2AFCC876F12DE363F1819778 8700C8123495632D8C7AC1FFC0FA7DAC19C0FD79988BA2AF769CF42EE90B470077235A)
output	0x643B2987F916D6284A05363E14AECC02171F11C0EFE772D036731E6D21320695B0B464B17 474CCE5F409938629673D2D039B087D30F0CF65F49A9CBE960C8DFBAA6D9DB89540CD46F202 C71BF52CD850111FF638D1137D55535C875B2D48A99A31E035EE74548CA9A18048123199400 91226B05E9E0B4BBAA90A85999B562F6A04CFE0ECAB5D829D1A92EB16120E12C67A50D65A02 C2A4C10B2250BF38A87ABD238E84EF9D2DCE087ABF65D2437F5623C2399FA3887F20FB48F54 166E2B527214EB68D0DE07E7232C7B811C4D2DB5242B313628416D50B771681A58A8AC4C450 1E9E5134E6AED676C65EB1B36C7DF4C01C9899CEED34FEDBF76AAC8E96EC7CFC

1.6 Phi functions

The "phi functions" are an abstraction defined in the document "Computational proof of Complete Verifiability and Privacy". They describe a homomorphism $\phi : \mathbb{G}_1 \rightarrow \mathbb{G}_2 | \phi(x \star y) = \phi(x) \otimes \phi(y)$, to be used within various types of non-interactive zero-knowledge proofs. Each type of proof uses their own, well-defined category of phi function, as per the list below.

- Exp proofs: $\phi(x) = (g_1^x, ..., g_n^x)$
- Schnorr proof: $\phi(x) = g^x$ (only used for ballots / confirmation, not in mixing / decryption)
- Plain-text equality: $\phi(r, \bar{r}) = (g^r, g^{\bar{r}}, h^r / \bar{h}^{\bar{r}})$
- Decryption proof: $\phi(\vec{x}) = (g^{x_1}, ..., g^{x_n}, c_0^{x_1}, ..., c_0^{x_n})$

While defining this concept generically makes sense in a computational proof definition (since it facilitates reuse of generic proofs), it might make more sense from an implementation perspective to define each homomorphism separately, to enable use of semantic names for the parameters and to decrease the risk of using the wrong homomorphism for a type of proof, which would result in proving something entirely different from what is expected and required.

However, given that the descriptions of the phi functions are used in the hashes used for commitments, their representation needs to be identical here for the proofs to validate.

The functions are provided below, in increasing complexity order (subjective).

1.6.1 Schnorr proof's phi function

Here, the domain of the function is the group defined as $(\mathbb{Z}_q, +)$, and its codomain is the group defined as $(\mathbb{G}_q, \times)$.

Context:	Encryption Group (p, q, g)
Input:	x the value for which we want a proof of knowledge: `BigInteger` $\in \mathbb{Z}_q$
Output:	$g^x mod p$: `BigInteger` $\in \mathbb{G}_q$
Note:	Use the algorithm defined in paragraph 1.4.1
Description:	"[[(1, 1)]]" / To be confirmed with Scytl
Suggested name:	`PhiSchnorr.compute(x)`

Simple examples

p	q	g	x	output
11	5	4	2	5
11	5	5	3	4

Real-size values

p	0xB7E151628AED2A6ABF7158809CF4F3C762E7160F38B4DA56A784D9045190CFEF324E7738926CFBE5F4BF8D8D8C31D763DA06C80ABB1185EB4F7C7B5757F5958490CFD47D7C19BB42158D9554F7B46BCED55C4D79FD5F24D6613C31C3839A2DDF8A9A276BCFBFA1C877C56284DAB79CD4C2B3293D20E9E5EAF02AC60ACC93ED874422A52ECB238FEEE5AB6ADD835FD1A0753D0A8F78E537D2B95BB79D8DCAEC642C1E9F23B829B5C2780BF38737DF8BB300D01334A0D0BD8645CBFA73A6160FFE393C48CBBBCA060F0FF8EC6D31BEB5CCEED7F2F0BB088017163BC60DF45A0ECB1BCD289B06CBBFEA21AD08E1847F3F7378D56CED94640D6EF0D3D37BE69D0063
q	0x5BF0A8B1457695355FB8AC404E7A79E3B1738B079C5A6D2B53C26C8228C867F799273B9C49367DF2FA5FC6C6C618EBB1ED0364055D88C2F5A7BE3DABABFACAC24867EA3EBE0CDDA10AC6CAAA7BDA35E76AAE26BCFEAF926B309E18E1C1CD16EFC54D13B5E7DFD0E43BE2B1426D5BCE6A6159949E9074F2F5781563056649F6C3A21152976591C7F772D5B56EC1AFE8D03A9E8547BC729BE95CADDBCEC6E57632160F4F91DC14DAE13C05F9C39BEFC5D98068099A50685EC322E5FD39D30B07FF1C9E2465DDE5030787FC763698DF5AE6776BF9785D84400B8B1DE306FA2D07658DE6944D8365DFF510D68470C23F9FB9BC6AB676CA3206B77869E9BDF34E8031
g	0x4
x	0x123456789ABCDEF0123456789ABCDEF0123456789ABCDEF0123456789ABCDEF0123456789ABCDEF0123456789ABCDEF0123456789ABCDEF0123456789ABCDEF0123456789ABCDEF0123456789ABCDEF0123456789ABCDEF0123456789ABCDEF0123456789ABCDEF0123456789ABCDEF0

output	0x8DE3DB1E8E54041EA83E88C31A8FE20BBA09162BE288F5D795FA0596309CCB597A969DEDE DC70C8B305A67C2CE8839A315FA368FF90800F7E2A3913A522B820ACF1F5E0F0020F1CA5E53 9AA082AE7ABD46B6B0354B925A2A9C9D8694050042C0DA2BAFBE122B015C35091794CB70EFA 3C35EFB4C6985A7D97F24BAC4F3A068AC18A11B80792A39434095D1BD32B7912586E66F28C0 11CBF15EF067F10B41C41549E80CF1A5985A643D95DD0AF87A7F014BCD880B1D07EF332231C C71AFE5242B6895A7DA331D3163AC635A42CE5795D936AB4A05431BCFD41582360E2DDE114D 2A974016572A6E0D36F19247EB560077EAC068E4BB1B9CCD85A5B8F2389148F8

1.6.2 Exponentiation proof's phi function

Here, the domain of the function is the group defined as $(\mathbb{Z}_q, +)$, and its codomain is the group defined as $(\mathbb{G}_q^m, \times)$.

Context:	Encryption Group (p, q)
	$(g_1, ..., g_m) \in \mathbb{G}_q^m$: a vector of generators
Input:	x the value for which we want a proof of exponentiation: `BigInteger` $\in \mathbb{Z}_q$
Output:	$(g_1^x, ..., g_m^x)$: a list of `BigInteger` $\in \mathbb{G}_q^m$
Note:	Use the algorithm defined in paragraph 1.4.1 for each element
Description:	"$[[(1, 1)], ..., [(m, 1)]]$" / To be confirmed with Scytl
Suggested name:	`PhiExponentiation.compute(x)`

Simple examples

p	q	$(g_1, ..., g_n)$	x	output
11	5	(3, 4, 5)	2	(9, 5, 3)
11	5	(3, 4, 5)	4	(4, 3, 9)
11	5	(3, 4, 5)	3	(5, 9, 4)

Real-size examples

p	0xB7E151628AED2A6ABF7158809CF4F3C762E7160F38B4DA56A784D9045190CFEF324E77 38926CFBE5F4BF8D8D8C31D763DA06C80ABB1185EB4F7C7B5757F5958490CFD47D7C19BB 42158D9554F7B46BCED55C4D79FD5F24D6613C31C3839A2DDF8A9A276BCFBFA1C877C562 84DAB79CD4C2B3293D20E9E5EAF02AC60ACC93ED874422A52ECB238FEEE5AB6ADD835FD1 A0753D0A8F78E537D2B95BB79D8DCAEC642C1E9F23B829B5C2780BF38737DF8BB300D013 34A0D0BD8645CBFA73A6160FFE393C48CBBBCA060F0FF8EC6D31BEB5CCEED7F2F0BB0880 17163BC60DF45A0ECB1BCD289B06CBBFEA21AD08E1847F3F7378D56CED94640D6EF0D3D3 7BE69D0063
q	0x5BF0A8B1457695355FB8AC404E7A79E3B1738B079C5A6D2B53C26C8228C867F799273B 9C49367DF2FA5FC6C6C618EBB1ED0364055D88C2F5A7BE3DABABFACAC24867EA3EBE0CDD A10AC6CAAA7BDA35E76AAE26BCFEAF926B309E18E1C1CD16EFC54D13B5E7DFD0E43BE2B1 426D5BCE6A6159949E9074F2F5781563056649F6C3A21152976591C7F772D5B56EC1AFE8 D03A9E8547BC729BE95CADDBCEC6E57632160F4F91DC14DAE13C05F9C39BEFC5D9806809 9A50685EC322E5FD39D30B07FF1C9E2465DDE5030787FC763698DF5AE6776BF9785D8440 0B8B1DE306FA2D07658DE6944D8365DFF510D68470C23F9FB9BC6AB676CA3206B77869E9 BDF34E8031

$(g_1, ..., g_n)$	(0xC604C21B2AC4FF7A8B36BA0CC73945F3E89F6643DC3429B40D1BD1530D9DDC7BA5ADF90F36870528AD72AACBAA8A5E7AAC28A0ADE36CB75A587C5DBC12F43E357E405947FD29902E5234844482570497598117 21F580FC96406B56E0DB6FD1B06541BFA6793C86271EFCCE7CA3543306A3BA31258EC373D291C69278E3E23714EAC8A43C211BBB25EDB444A609EDE4492F10FBC7D1A9949305DD9B9987BD980265F501AEE840229F31478F19C6AB4AFE0B35B6801E859C83A3CF7CC7D3BFC9DF7D8DEB78FB0E0F8DBE64CEF3D97DAA0FC0A977F57968DF291BB80822DFCA62BED1CB893E7D90D485406093D8F3457F7407C7374CE52AA44E5D7594C6AD90B58, 0x16D328A54F725AC93184F49C7FC740FB4C48C39D2721FF29DBA190379C690DC9256BCFCA2B83E3C01AAE10133907F108BBE693312E07B5F5F2C53D31BEBA3520785E2760D11140635CD89BC40AAD75FC349D5771D98675ACC8D7E7D8FCBAC30780D8F4A7AC6EDB4A005ADC0909E72013CDF63E764ABB116B259B225694BE92572BCA07723C44B4538CD025AD38E546D7D4DBAC16D378AE1805D5C331CD4145AF3E580F313AF8139FC5652EE8CE078E97F334DA0820B110AFE424E5948CF7A99040D330F2B90C78A902DBC114E23A062B0801B6004F7CBF9D7DBB00C4A0FA6B2F54B7D44A8066095F20B2636183C7BC3BC43B95595B1D134A48524946AD738F25, 0xA10D5EFD651D1383C2C6CE3A83AB5337DC4225AEA6BE8C796C822F6BB284D75EC2EC6747F431310B3BFBCADEAF61C08246B3B0D7804AC01304FA318AEFD755203B72D3E1C87162F3419202693FF044D0E20FDF86C90A3D774C05B314775128BA9F5926A0C663035940BA4E95280566474A3FAAEDB8EDE0EEF70D26C6652271F4BEB790286144EAB2B4C17E7B2499B76643A2CB32D8E71BE8A58CFE86522EB38E4EF5D92335689DF01ECC20FDE0B14C9E7781A3AE43DA82BDB7F30548C4251B3B4A1A74D9D09F65F86DFBD26F5B57339EAE2AFCC876F12DE363F18197788700C8123495632D8C7AC1FFC0FA7DAC19C0FD79988BA2AF769CF42EE90B470077235A)
x	0x199A56BDE2D14B178711FBB69B69643A858CF27AE6CF16BA2906E906EC6323B0869CEFA3C5B0260CD38CBDC4993D8B5ACB53675DDB0B5B1EBCAB62AA737CF5BB887C71EE8C938DA1AF19A5B7FB66DD1DF601E8FA696437062AE8ADF2040B7B8C2B39F3FC3BD60EF2EBF596698EF290C8C07A68FD8E8DBCA94F2C44358B7C9077D32D941AA5AD4FC8D3E1B69CFDF65B14337B59F054F8C0F17467EFE9F0D7103C6F88E0806BA1575A94051776460AA01E419099C04B49C5E7E743D457325928A1F2CA58897F83BFBBF682434F0BA05A9F65D841E47D71CD930410FDACFC9D2DF2B46D937822E31F42CAC2832362749F9F68068AE4D4AC6AC8507816EBE24BF0A1

output	(0x1029DCA5510BA6FF0DC3D1A6F6A712DC89ABF0AFDE6A254D344960DC8F5328CC1165B1AC14D65ADE3F0364B6CD895AFB951BE7F2178C680D8540E5B3FC66B15930B8B9E2394B3265DA7D810DD0F6BCFA9A1E1DA7373D4330E47E37AC191A9FC338A6921B1F3E372CB61A10B03D478A8FEFC009ADC321E11D2832ACB55729F0ADF51535EE9FF499AB4CC945AB9DC762117970CA6E04C479C309D64B7B118B81C4E4D280AC8BE2CFA8D0A8B0665EAD62EDE6D33A89E8B54DDAAA4D4272A628CC0043A07ECFDBBA736BA748646877AE7D050CD934DC3142B2555C1AB81ABF54F6C0D6A3B289349A78A0114C3D769545B92C4B2F5C6A13A25AA70195174A2DB6C2F4, 0x306F666263795FFA92EF7969B1E194272DEE0C45CF95112F9B884BDE18EFE65893ABABC1839BD9D5C27653C404555243893A6FC33925925059681C0B68C8FBD4C1A61C73BB4E8108CCF7F7BA7CB978DE4CE79E120062E2BACCD0BE79F3D921FA45693AC21B6DE73C8BFFD4E59C53518B4AEA0D908AB450C18F75A647E72B5B9C75A844393D29551052E13BD42B422920C77CB6AD20A10EBCC0E9C0715EF0787E569259AC28CCBCBC38181D2290974525583361E6341ABADC0C6F0081E7712935D02CC1FC7D3F789B5A41AF26F3B764A7D203D8D16EDBADF7E4129CC321559A499DEE0E1B6C218783FD57691F6B840942B2B03F5D71B8FB84D2D8B42C34C4E8C1B, 0x773524805C8F76217694F18E710F076C8B10418DFD6AD778B11A8534DD4CEB48F0EE1B77B3E4F9D361D6F75A69F9EBA35869C7B587000875C96F8F4AA12F157928C0ABDD27F014C2B5F02D991DBB1191C26A04A4EDEA7EB2B2D362CD88E1D27223261227CCAC3FFCDB053138D261D9AFBD6D40F20B6964D22CCCF17F60E0635D6FA965368EC32970C4D0B1C1994481279C66A6F6079DEB993EF0BE74D9B398DEA71F1833F35BD8A91846383EB379704CDC1818F526220156439E9059FC376F48FFD17315B653A5AF30FB285742E55E293A50CF72240C6EE2FE9358CE170F884E064612C4A8A781AB8C5100CCDC87777F76D26BA4E346F4A09D6C6E8C66E018A1)

1.6.3 Plain-Text Equality proof's phi function

Note: while the computational proofs document only specifies plaintext-equality proofs for single encryptions and defines $\phi_{EqEnc} : \mathbb{Z}_q^2 \to \mathbb{G}_q^3$, Scytl's implementation adapts those proofs to run on vectors of values instead, that is $\phi_{EqEnc} : \mathbb{Z}_q^2 \to \mathbb{G}_q^{2+n}$. Proving the validity of this adaptation is beyond the scope of this document.

Context:	Encryption Group: (p, q)
	Generator $g \in \mathbb{G}_q$
	Primary key $(h_1, ..., h_n)$
	Secondary key $(\bar{h}_1, ..., \bar{h}_n)$
Inputs:	First random number r
	Second random number $\bar{r}$
Output:	$(g^r, g^{\bar{r}}, h_1^r/\bar{h}_1^{\bar{r}}, ..., h_n^r/\bar{h}_n^{\bar{r}})$
Note:	For each $i \in 1..n$, compute $modExp(h_i, r, p)$, then $modExp(modInv(\bar{h}_i, p), \bar{r}, p)$ and multiply the two values, modulo p.
Suggested name:	`PhiPlaintextEquality.compute(r, rBar)`

Simple examples

p	q	g	$\vec{h}$	$\vec{\vec{h}}$	r	$\bar{r}$	output
11	5	3	(4, 9)	(5, 3)	2	1	(9, 3, 1, 5)
11	5	3	(4, 9)	(5, 3)	2	3	(9, 5, 4, 3)
11	5	3	(4, 9)	(5, 3)	3	4	(5, 4, 1, 9)

Real-size example

p	0xB7E151628AED2A6ABF7158809CF4F3C762E7160F38B4DA56A784D9045190CFEF324E7738926CFBE5F4BF8D8D8C31D763DA06C80ABB1185EB4F7C7B5757F5958490CFD47D7C19BB42158D9554F7B46BCED55C4D79FD5F24D6613C31C3839A2DDF8A9A276BCFBFA1C877C56284DAB79CD4C2B3293D20E9E5EAF02AC60ACC93ED874422A52ECB238FEEE5AB6ADD835FD1A0753D0A8F78E537D2B95BB79D8DCAEC642C1E9F23B829B5C2780BF38737DF8BB300D01334A0D0BD8645CBFA73A6160FFE393C48CBBBCA060F0FF8EC6D31BEB5CCEED7F2F0BB088017163BC60DF45A0ECB1BCD289B06CBBFEA21AD08E1847F3F7378D56CED94640D6EF0D3D37BE69D0063
q	0x5BF0A8B1457695355FB8AC404E7A79E3B1738B079C5A6D2B53C26C8228C867F799273B9C49367DF2FA5FC6C6C618EBB1ED0364055D88C2F5A7BE3DABABFACAC24867EA3EBE0CDDA10AC6CAAA7BDA35E76AAE26BCFEAF926B309E18E1C1CD16EFC54D13B5E7DFD0E43BE2B1426D5BCE6A6159949E9074F2F5781563056649F6C3A21152976591C7F772D5B56EC1AFE8D03A9E8547BC729BE95CADDBCEC6E57632160F4F91DC14DAE13C05F9C39BEFC5D98068099A50685EC322E5FD39D30B07FF1C9E2465DDE5030787FC763698DF5AE6776BF9785D84400B8B1DE306FA2D07658DE6944D8365DFF510D68470C23F9FB9BC6AB676CA3206B77869E9BDF34E8031
g	0x4
h	(0x9BA458D54E947346863A211B49259DC190A830AD228C2D7920BB5BB4ADA999BDBD423C77BC577D58FF5016755E049CB09B208C95859327937996 56F4AE021966A7BA5AAB654730E78C336CA5485CEFEB41AF8047F922CAD2935EED717F9F09D7845C22D2D9F87FDBCA05E489FFBCFCF0ECE636916E670535D3D940D6574BAE7F6684B48A583CFBCC6D8983EC7C1099617317AC8B203E18CA092C1CC801D266C718337CF4D9447529B8993359D335C2DF895ED143264F5B3339B62EC2F5E52E21EA9FCB84C9FE122D05D45F8569581975353485206F95DC774D5D64415E1B37A8B1BB1E13E0FA0F46827F13F0204B817AD43FF322B41F2136159185D558621836, 0x943DAC1F99FD5CD4C917031E3B0BE4BE8544DEDAF7EAFA9878BFE92963CFC7B3B26AE29810D426D0F685F700687701EC65867290ED5A812DAB1B8A7B4DAA851BD92BEC1E65DF15C5AE48865D6CF41377F38017C7C6FEE89DE47ADA5704D2B481386CA8AF969CE340D33FD2C5A9B37918DAB734A93BC4FC46CD9A534BC116269AD470DB5EE8953A40233824AE99E17A989EE512881D275686AC26873AA22AC31B845C4E4F749871F18ED4902405E75DACAB1509AB6C02EC9A9F49991E11F04DAA789415CA9DB6502097B68B8953C614058B6B41D30451AC5E74B9B308B597BA6F907735CD3297D449041DDADB971F33229E5D0CCD6DEB67ADF20FD3045AC5556D, 0x3D9BFAF9A92DA7D6B281BE41FEACC750F1D43AF09CF9EE519524E85041D668566642BAAE264349860F0F6CFBBDA9CD7A2AFFAA478E56F3BD18DB7BE3186F15793B5507BCABE0C97950A771F5BF0AC6A2C64968AC62E83D578B8E371B2CE279480398A9E269BA34A49A0F465DE58DF12D003ECB7CB16AFF26E5568B7E0105F9F77D6C566542EEE051BCD9C7D42C3F45E2C2D94C82EA6B9359B04444BEF6193958A45CE92F10395852C76C28963ECAEE0A25072B6AFD1635E64AF61095EBFC4B59C82F77C31E89B1A2B084B3BD1D2D3FFD6A4CF68BC1C72890A64802505434BD8ADFF86F24082F53D647044C5C905343AD619B789B4DB4F3A02D0C268662868EE2)

$\bar{h}$	(0xA4015779EC703DBDD7EA96FB5FE2AAC5F7ED97A31C41E06454054498F126DD65A910941731A649D3D36DA750C2E5DD79C42031AD7EC13184522CBD08F377765A28516AA539BED2D5AABA4E87AEE2691FBFAC658A7991698768FFD36A72FF078379B350626727FE76EE0CE8F8368EAC1ED620B7E120A67F5C47DBB316ECB41D1924417ACEE0CC4F0CD31C7FDF9F913FA5CC0BE3F2D64437EE2E8E242E8E0E6C82CD7DC77CFA4BBCF51CF3C69707149A0BED43AC39A6F04271A0E683AF4520DC9060FA8F247A99BBF3875D362520C498537AC75A9582598F821A575C1CBF669F5F2656335C55EF57BF2CF14ACEC7AEB53D6A384C1A737485185AE8C615B94F9C06, 0x81B73AAE0DE6AA2EF2CAD306973C78B575E558B4AAF2498CF9BE836D17FC6146D39FED1F6A94CA6E5CA7210E386B8033E3E6F7BF2CA4E86C5FEEE399B259AE1854C90BEEB134E050E8703AF817BBD654D04A2D7DEB3F4FEC6882A4F2E8D62F6B9D085D05983DDA8FAB82BB3CD67E1154695A1378756DCC73B7FFC948AD5AAC3B16683C2C694585C55483E79DE979F1FE7FA1AA57DF1AFC72AA949A9CFB3FFFC1610EA1DAD2AAF340E74DEFE176A18E7B2A9F1A6318F3900E2D2DEBE60E76AAA743FC8A955B533A815CFC6C28D09B09E82F7348DA50300FB86F4DD477C1719E3C71828DE09DF73889C746D4F6455018893770E683B65BB3A846FFDFFDD49F0F2, 0x5E27281F4D02E5F1192577B6FC2689222E859B957233CA3C4B5F89365F0F0DD11639818441568D0E4A77B865E64515B0AA424FA39E63F9E7E2215084DDB942B8E10404056C17CE1D5FD62A7742F97834627D4B4DE833EF1A81E5820F2916558471E822ED5A6242503F1A3C80BF4CBD43E321DB0C6DFBAB947A6D218DA1E5EFF374C4A2639BB76F35DCF91F989802C73B2739476519FF1BACBC7E51D936B68C2702A2760808B3FE01897FA931B0C2E73F6C77DDCE58EAE6EFAB005ECEFB1343C1DFD9AC58D2A6C1F52BFA70126C7A536EECE1F104CB244F0247696706D140BBDF635431C9187C64D9CE2188DDFD98AB8E53832D6054A6BFF93116B3207825A2C4)
r	0xC4953A41F8120D390E396877C5EA37B36AD9AE89C61F07E23F0CFAAAFEC93536DF0E5D14F929C9787CC26958E73212CC4E90A0855C945BB7F3FE05127DA8B78A7EF7206202B2105E85022D144E2F1D36B075689D0765DFA9A726F61B76A986C967ACB3C5F46F16B63FD5F486B373355747ADB3ABE1226FC3274959AA5BB11DBD527FAC951224E17B8D10995D1187918198B344E9FF2CC78544C24A1C815BCBE6A01B2DAED0087CC00496834F1B8AE7D95DE74CE951F0033C5C89934D6A3EF2349E16C06DEE295BB27147AADE1E217A06DA094E370BB848DE4CDB25722C0098ABA6BC02C4FA973BB91709341D74BC9E75E148AADB9B3683FBB25A0AAE70A796B
$\bar{r}$	0x3966263D0FB9DF563C51E4752DEDA75382466C676864E0849A24613525FAB7F491C6184F810E6F74A8E12147D5567E92053F900808EDEA4595AE46E7126AC305E5FDADE112A94A3E7ED9ADD5984BBB107696E69ACE4CB051906FDC2668EE7D5EF8F8961DA9CBC3F439315DDDF3A8461934A56D612971BC3933BAF1BE3C93A47A806F570ACFA3802D54919B9F6F8BB90A2B0CB8188FF91B7528826D88F6AC62C2F2401F5F3EC8A91ABA088BA5C83FA8F6E9130F6338D5AAA8D9092B81C0DD14F4B86B89608EEDF43C10112B0385E5E7FE021C31ADEFDAD120331679DB4145405FC30D89AE743F86D6F3438177BA1E77E58DE85A4B0AA337D2EC0B14CAD55231AC

output	(0xB1C7F0CDF9466ECAF195EAFCFEB0CB838D73AD28388AD592E278A24454DF0A9FFF53E A5F30BFE33408B978812A53A682916D61CCC03A6BE9F6DF25182EE4895465DFFD7F8EF00 11E54AE7C206C4FC15019EE3B47E6A4F3DEA8F66F6F822FBE2C57739FC6A0D3D67FBC072 DD11614EEE0B8C2CA0B744FD26F3CA3CE75F06E8CD685A97CEA9F14642BBD6CD3C5FFF75 57FC67FA74882B4701C2D9B3F2F3472B4481A7282EF1F2147E438DC6A08460354213ED2C 6C2F8E89D191A936182E5AE12EF1C628B82258CE19084242A426A3E14B0BB972A2CD147E 3437E869BD2A9E92CE1FD5B216A5589151FA03D9F209C27B813E98C3AC248D4565262A2C 7528EBCA4B1, 0x809FA5FFFDA1AF03B646A464479E4F8BE4A4E4A917AA280D24C81882FBB331C6FC1DB3 8B3182B7D1F4A127BB74CDDB61EDBE7A5CB9B0E5ABEB36C3D4AE75C59E5D758C9D7CA220 078FD685070BB99BFCEC3FE7E61C83C283C7ADEC8F6C12F65E411F5D20A340DFEF02BA08 B7332622DA979FBBBF3F7E927D9F58B1919FE3879F56698BD29EC3831962FD4109650D8B 1ED5FB52FD89CFF811A733A4C10D182EF4B56483C5859F3D09C1B58EC22A21EAF0872626 B43B1AC9F4313D906F9FCDC3281FFF843FCCC1CF9CE2EA85836DF8907329A7FE8FA3163E 962D6681A772EFA3CAD8068AC29A67E35B4477DA043F2F66381E2455959C617EF2C40A17 CBFAD11818, 0x2DDE95EABE125C180208D05BD3505BD53C3969664E8ECA6B15C3DC994CAA9C025F7841 D307D5A814F77BC48C1297EF314427F6E9C103751617B1EE31343F13CB67657919556731 61AC5BA80FFF5CC20B45A63550E7365174EC20C119B801E258CD08E3974AEF4DBC9F1831 6461C785D2DEE51E6CE6ABDF7DD464124ECEC3AAA14030F03E469B3BF5971C0163DF928A B7197307EF551226CFBABBDAEF646C3808977CCB27F2056B5543EB78B7FA8E71BD3E09FC C2A35B971D30F4817D12632A437F3927C7911FDFC48DCB0C7310D374EDE628FF98EA93D5 C333F0387C7FB52EFDDC48CD2F33D0781A26ADBCC9C7E753EB596458D39ECF6FCFE411EA CB2C8BF641, 0x806F33AEE32CAE9745B26BA0EB8273206C569145218475DDC9C133FDF0E86AF788A998 9F7E3CC1B915DF4D676B85E318941B2B88887000925034DBBE5C143E6D98C5018581B624 72B5176DA6A5482289A916F54FD9887F6DDD043F00C8D3144B032877D2CA300B07A60E65 50667A5E0AAB1F9BC0AD8AFB394E5A46E70C130A038CEF3DDD5233CC530F3EDB2D200F34 F34C72CCEAFDBCA5D7D63E42FBC1350A07E7A2C475F0B1867A5719B408D6E8C9B5FD0528 A2CDAE95DDC66198974E69BD682A82711A3ADF49E882653977F4DB3A8ACDCED5D45EDA6A 628AB1EAF9064613E288C000C2DDC96AFF69DECC5674E402DF47BA0978255ACD206A0583 2F10B56BA2, 0x8E72B4612DDD62F81127C703734FD6A354B99441110B626F0342A905AC3DDC94A6248E 2E8335B29DBF42E72619C0406571126953A6D283E7BDDAD7B95D5B201C953FC7F89BE149 05F16439EB1F6CD98DBE04EB02D514969E51B6FACF063D280F92C26BA983015755FBFC49 C041CB0AEA96523A5E968B72D0EEF12773094E21502775D9C75CFB1448B0F73FFCE828A2 FF0D896999BC9FD20A463DFFAF42BE4B66CF15C0346B060D004AB5122F8DC02BFEEA002D 81C0A91462BB74D70B9EE126C952B18748FBA22D39F21BF1ACC814A2EC8F9B45D00888EC 373BD573E52BEFA14BC7BA4643DCFA5B2B60D2E7CFB1B300EBB8A844683DE5DE3CE030B0 3B148A132E)

1.6.4 Decryption Proof phi function

Context:	Encryption Group: (p, q)
	Generator $g \in \mathbb{G}_q$
	Ciphertext $c_0 \in \mathbb{G}_q$
Inputs:	Keys $\vec{x} : (x_1, ..., x_n) \in \mathbb{Z}_q^n$
Output:	$(g^{x_1}, ..., g^{x_n}, c_0^{x_1}, ..., c_0^{x_n})$
Suggested name:	`PhiDecryption.compute(x)`

Simple examples

p	q	g	c_0	$\vec{x}$	output
11	5	3	5	(2, 3)	(9, 5, 3, 4)
11	5	3	5	(1, 4)	(3, 4, 5, 9)
11	5	3	5	(0, 3)	(1, 5, 1, 4)
11	5	3	5	(2, 4, 0, 3)	(9, 4, 1, 5, 3, 9, 1, 4)

Real-size examples

p	0xB7E151628AED2A6ABF7158809CF4F3C762E7160F38B4DA56A784D9045190CFEF324E7738926CFBE5F4BF8D8D8C31D763DA06C80ABB1185EB4F7C7B5757F5958490CFD47D7C19BB42158D9554F7B46BCED55C4D79FD5F24D6613C31C3839A2DDF8A9A276BCFBFA1C877C56284DAB79CD4C2B3293D20E9E5EAF02AC60ACC93ED874422A52ECB238FEEE5AB6ADD835FD1A0753D0A8F78E537D2B95BB79D8DCAEC642C1E9F23B829B5C2780BF38737DF8BB300D01334A0D0BD8645CBFA73A6160FFE393C48CBBBCA060F0FF8EC6D31BEB5CCEED7F2F0BB088017163BC60DF45A0ECB1BCD289B06CBBFEA21AD08E1847F3F7378D56CED94640D6EF0D3D37BE69D0063
q	0x5BF0A8B1457695355FB8AC404E7A79E3B1738B079C5A6D2B53C26C8228C867F799273B9C49367DF2FA5FC6C6C618EBB1ED0364055D88C2F5A7BE3DABABFACAC24867EA3EBE0CDDA10AC6CAAA7BDA35E76AAE26BCFEAF926B309E18E1C1CD16EFC54D13B5E7DFD0E43BE2B1426D5BCE6A6159949E9074F2F5781563056649F6C3A21152976591C7F772D5B56EC1AFE8D03A9E8547BC729BE95CADDBCEC6E57632160F4F91DC14DAE13C05F9C39BEFC5D98068099A50685EC322E5FD39D30B07FF1C9E2465DDE5030787FC763698DF5AE6776BF9785D84400B8B1DE306FA2D07658DE6944D8365DFF510D68470C23F9FB9BC6AB676CA3206B77869E9BDF34E8031
g	0x4
c_0	0x24ED12E8F084AB2EFBDF284316AF1A1CE68B77611A641C6AF3CAC809CD949403953DBC91E1D3FF7885F677BA2E7973CFAA8B1F572E832DBEAD1D87C704649E30B7CC75F01111F65CABD2F97B8CE86EDF1CD8508055A6DC1022661A773D261BACF5490FCE85BE4703A9FBFFA8575FAA70A60C6BA851F7CB8F63B438F3E0C231796CCB888BB8A1E34C335EC0FFB2D5A0436F8275FEED866039A9D770E0485A20C3F3FB9D0AAB270246CD75EB37EE0A4B4D755FCB41381C461BF731D23FD5879769BEE7A81CC58753E01DB86A826394BC473DD77FDF4486497B0A8DB041887EAE752F78BD7211A03E1B5D51BCE86197D38423D5280BA23D887839C12919E037E36E

$\vec{x}$	(0x1300E056E34C0805F67DFD8342FCE696B6DE7358276B7B2B174A5E248BD4866A316D1 E08610C7093C431A5D2FD7238B7C78764B9789C354158BA5C28049A52B7EDD4643D6C1A8 7DB0B00E10D28E2F72CA377778B7992F0D7D9559706D9D5A24D563E574C92A3EACC14E83 FADC42191715226D8EAF6695966A56BE1834ED8DFFDDD66617E7B3059E0703833B54A6AB E6330C04BEE73B5A5AA7325A3DD860E270CC9713884D6CB261DCA4A2D66B40BE8868F82D A19A50C95FD3F4720DB016ABAD8B441BD27885DA923140F95CA3EFA6649F4BA4F80CD598 F7457D53E7621F9690F6905293E2A2E4EB1F25A8FEE2AFD18BD65311FFE91DF03F0AA170 6DA96FE7BDA, 0x5B0D92C9A033B148B9C38A5E824315DA7EE2F817A85C79B7A1CD4E9E4C210F1FE2A726 81670759F634BE4E6417B9FB790A44B7B9343C6F69E66D8D938FF38FAAF8D9B05480D4B2 44D9909000EB869BFFE3AB0B611247B157EEED0E983B162E75E23C5761D3B94793C89E67 DA2E3F090C3435E6C2B9FD235CB287934AB292586B7AA8F9A925CD96E52EF22F7A3D34E6 B658AF50D81AB8674F8ADF11D3ED072BDFAB4A2A7345FD1D61EDF40180CBA085490EC72C 5A3CA80D5A330B9BA9FD52B33B8595D725B72723FB7BFFD394CBB765E84D7DD9840810F0 121B0972D2EAC7F73194070095D7B8319353C9C65E77CDC3EBCC7AE7A7A65F0A73A78E0B E1707823F1, 0xC31257C78D3F9ACA34CB5F5A990D2FFD16066F4EE9C3C05F28136A6C9F1025E544CB17 4497D07AEA79EC7A05F05033024D6C8390086127A1C23E1F90D0C2EE199586B8CCAB8D4A 32778B1A37F3803EE75E1E7315B9A8C932178DE69BCC76D0FCAF8777D83BBCB089EB772A 721114D3597CE4297A2BD1E2F0FC1FD13DBE4B1D73B8A3C5FC6077C4EA85705E8459ADA0 1E9E2DA41A48F3CAB8F19975189362CB304A786595F2120BCD756DB4B98A02EB8532194F 570C947D870FA5E7BCA677D5FFB754432FEDEDC881BF9177DEDC1855D93D3FFEC5BE2E05 597683A800CC7255A90C523CC041A1F293ED24D4E0354CD8D8AD97B025378AC1FB8B3434 D803835AD)

output	(0x9A0F502D9BB5A28452CC1E41EBE3D61E3234F959B197AC7E5B8D3B2EE7437AFD95512 E90DC5004E869EFF52F5CA549AF9A21A95ACAF28A152C0C62D8D38AFEA7EBED154D60A88 08D133B5436BF1BEEEC0C24973ECE6274D96681F8FB9F00D3AF34D849FBDDA91EDE035F8 081488069FC618121F7B7CD39151C9C9DB07644E28113B1C01FC4682621B258711B2A47A BB5204D99DDAC090CA05108F8A5B57D93E278057F0EF564D7CDA5D419B0176FC3CBBD4B7 8DF8D5FB87F6ED15F27595D44C7982C3713BA8A60CBDED12E15AB32101512ADF6E043539 60F16A04BE368DC3E6DCD4CD17C476B9B24B5356C3AE8838C36FD65D541553A08CB3DD4B 1115A91B38B, 0x2591FA9CA5AFDC5FEC22F87DA3F28EC56D5691A5B622390BFCC6CB0CD9B351C6DFF8B8 2A2699B10AEC52CC4870F05BA67D377CD49D04E29B9843CCA221F74FC021F768462A272F F9431E4D0C0082EE2FEA8D7A4568A69702DC04379A932E8D8B274F953E37DFBD326DC581 C72FB41977C08EB2D8DF654F5E91C796DB72ECFD1EABF524CD33C105DE01925303510851 9610BCB582D40D6F3CCD0329A5F79F92CF20AF4D4F301C099C76621C42DBCBE0AD10FF6A B051F1D5232F296C14C7157F9817292DE007F15CEB6D90297535E31A1D8A5C5C8B67F4C4 2B5EDA6FD2C5B55BDB60B6F30CA363EC41C3B4EAB169F1CA06B12D7308012DD2895C9D56 C9C10B528F, 0x72AB126A0536BC92AD3D4B5CDE4389CE57477AF08BE333E508A9E8F90060549F2B723D 8E2072268F9E79A0B9CA4538646716867418A6281EF8122D00461E97D72F4983F4BB70C1 834153B10C9306DC88161549C41FA97BFE9EDD86D040C27D8DAC034826F199238A468774 FAF6F1E6943A6B5E6C26729B95DA2285383C51507B6E964D87EAB4FE8FAF003A585D35B3 7D45EE8B5A33245FA9CB9D4202E61075B559BF0E2A1170EA6CEBDCC803EE19C6F62F9AAF DDAE49931E969DE719AB97A295A5A612A3B063FFE7232F899A3D74AABB111CF5C760C411 12F28DFF9044CA38DEA87322B93EE3B43AE77CE7BA8F46CBD038E27FCFE5956AB41E0539 EEF2AC2A7B, 0x6AAFF63E208C83294CD09B4DF0F01C08A18ADB5314EC54774276C324C4F7D59044B55A 6991B5FB9317093B41DE0BDBD45853682D0B8E66DC4636373D7DFCFCE1889DEFB6564CC4 B878C6F243F2FCC03DAAE3A15BDEE23D14E582A022A1D72F11F08FB7DDC3881984354607 B9BD6B7CB40F14036CB60A5355C77B5C497466B477AD4379CC2F9AB7438868DFA6B5C868 F31A6F61FA90F1148A2A38FF1CE28DFC4FA49C13A011DE2B67F2F3D0DC47E49C227FB329 68A8A753B18279A7626FEB081F5B7791E624E52F24AD03F673E824F9AF81BBFE25A88844 45D2BD81F8F242B54E862EAA047207E5F8503C2485FBB61BB75C9F9F337A4640C96F1AF8 2B932D1B51, 0x93EADA2681F8B8EE3184E0D1BF12791F5DA6F9AB822B6BB7A0BEBC5F249AAF3216A331 D16BF2D2E04FFA0EFADD39E92CE88B8B9010A480021660AEB5E7E2B5256883C63F47C1F7 9BC5433BFEA76BE8BC7F609C185C229582A8DC3D51C22D4C23AC6CB30865627CC4157195 CB877036F8BAC373DF08EC5E84811520AC275F75277F4A5CBC50342B5A67998ED41A2BD6 6340A83C5147A487A1476AEE3C1120C37B22B313A3BA39ECB72FCE7D53CAB8CCA621CDDA 51D6A646E5CC1DFDCA82618658290E6B280CF237AC31F311765F21BCD56F3D879861134E F80130A4B6FCF5B99D772427460C5C1B7075AB0135E7225D78D86B2CDA1BD6A88C59369B D0B489C82, 0x599B1B886ED01ED9F77DA281FA851F86E00631AB78732F9AFB6A839DFE96C3E74F2579 1F1FC75E395EA2B4A87550FAAB06235D1744BC7D2432F4B43A578AC7A559CB2C7F25D628 1666C6F83520D84A710F632A1718D96FAE78078CF7A15673D2F2AA8905CCB8CF0115E535 ACC3D512B3F98C078EC2F1EF53D79776167244C57AA126190C08057835E7D3CB5307FDA1 2475776416EE84040ADCEDDB2050410F74D3223BFEB434F54C919041D40B6AB583CADC48 9078C8DE041F30E02DF4823DB440F87CDF365D810CA0280254C02079C9263E693326DFDE EECCA1277606FA76502A70EC79D35A6754BFB98DEE70BAAFD8027B43635EB868A7B206AE 16E1B855F5)

1.7 Bilinear Map

Some of the proofs below make use of a bilinear map $\star : \mathbb{Z}_q^n \times \mathbb{Z}_q^n \rightarrow \mathbb{Z}_q$. This function is defined by a parameter y, as follows: $\vec{a} \star \vec{b} \rightarrow \sum_{j=1}^{n} a_j b_j y^j$.

For the sake of simplicity, the algorithm provided here takes the values of a, b and y as parameters.

Context:	Encryption group (p, q)
Inputs:	$\vec{a} = (a_1, ..., a_n) \in \mathbb{Z}_q^n$: a vector of values in $\mathbb{Z}_q$
	$\vec{b} = (b_1, ..., b_n) \in \mathbb{Z}_q^n$: a second vector of values in $\mathbb{Z}_q$
	$y \in \mathbb{Z}_q$: the value in $\mathbb{Z}_q$ that defines the bilinear map.
Output:	$\sum_{j=1}^{n} a_j b_j y^j$

Require: $|\vec{a}| = |\vec{b}|$
Require: $\vec{a} \in \mathbb{Z}_q^n$
Require: $\vec{b} \in \mathbb{Z}_q^n$
Require: $y \in \mathbb{Z}_q$
$result \leftarrow 0$
for $i \in [1, n]$ **do**
  $result \leftarrow (result + a_i * b_i * modExp(y, i, q)) \bmod q$
end for
return $result$

1.7.1 Test values

The test values can be saved from here: .

2 Arguments

2.1 Single Value Product Argument Verification

Context:	Encryption Group: (p, q)
	Commitment key $ck = (H, G_1, ..., G_n)$
	Public key $pk \in \mathbb{G}_q$
Inputs:	Statement $(b, c_a) \in \mathbb{Z}_q \times \mathbb{G}_q$
	Argument $(c_d, c_\delta, c_\Delta, \vec{\tilde{a}}, \vec{\tilde{b}}, \tilde{r}, \tilde{s})$
Output:	True if the argument is valid for the statement, false otherwise.
Suggested name:	`VerifySVPArgument`

2.1.1 Pseudo-code

Require: $b \in \mathbb{Z}_q$
Require: $c_a \in \mathbb{G}_q$
Require: $c_d, c_\delta, c_\Delta \in \mathbb{G}_q$
Require: $|\vec{\tilde{a}}| argument = |\vec{\tilde{b}}|$
Require: $\tilde{a}_1, ..., \tilde{a}_n \in \mathbb{Z}_q$
Require: $\tilde{b}_1, ..., \tilde{b}_n \in \mathbb{Z}_q$
Require: $\tilde{r}, \tilde{s} \in \mathbb{Z}_q$
$elements \leftarrow []$ (start with an empty list)
$elements$ += $publicCommitmentToString(c_a, p, q)$ {see paragraph 1.1.3.3}
$elements$ += $exponentToString(b, q)$ {see paragraph 1.1.3.1}
$elements$ += $singleValueProductProofInitialMessageToString(c_d, c_\delta, c_\Delta, p, q)$ {see paragraph 1.1.3.4}
$elements$ += $commitmentParamsToString(H, (G_1, ..., G_n), p, q)$ {see paragraph 1.1.3.5}
$elements$ += $zpGroupElementToString(pk, p, q)$ {see paragraph 1.1.3.2}
$input \leftarrow stringToBytes(reverseAndJoin(elements))$ {see alg 1.2.3}
$x \leftarrow RandomOracleHash(SHA256, input, 0, q)$
{Taking `RandomOracleHash` from alg 1.2.2}
if $c_a^x c_d \neq computeCommitment(\tilde{r}, \vec{\tilde{a}}, ck)$ **then**
  {Use the commitment function defined in alg 1.5}
  return false
end if
for $i \in [1, n-1]$ **do**
  $args_i \leftarrow x\tilde{b}_{i+1} - \tilde{b}_i\tilde{a}_{i+1}$

end for
if $c_{\Delta}^{x} c_{\delta} \neq computeCommitment(\tilde{s}, \overrightarrow{args}, ck)$ **then**
  return false
end if
if $\tilde{b}_1 \neq \tilde{a}_1$ **then**
  return false
end if
if $\tilde{b}_n \neq xb$ **then**
  return false
end if
return true

Real-size values The JSON file with test values that should result in a valid argument can be saved from here: .

2.2 Zero Argument Verification

The Bayer-Groth verifiable mixnet distributes ciphertexts in a $n \times m$ matrix to decrease the volume of data needed in the proofs.

Careful here, the indices for c_a and c_b in the statement start from 1 and 0 respectively. The vectors are completed with the values of c_{a_0} and c_{b_m}, taken from the argument to obtain two vectors of length $m + 1$.

Context:	Encryption Group: (p, q)
	Commitment key $ck = (H, G_1, ..., G_n)$
	Public key $pk \in \mathbb{G}_q$
	Number of rows m
	Number of columns n
Inputs:	Statement $(\overrightarrow{c_a}, \overrightarrow{c_b}, y)$, where $\overrightarrow{c_a} = (c_{a_1}, ..., c_{a_m}) \in \mathbb{G}_q^m$, $\overrightarrow{c_b} = (c_{b_0}, ..., c_{b_{m-1}}) \in \mathbb{G}_q^m$, $y \in \mathbb{Z}_q$
	Argument $(c_{a_0}, c_{b_m}, \overrightarrow{c_D}, \vec{a}, \vec{b}, r, s, t)$, where $c_{a_0}, c_{b_m} \in \mathbb{G}_q$, $\overrightarrow{c_D} \in \mathbb{G}_q^{2m+1}$, $\vec{a}, \vec{b} \in \mathbb{Z}_q^m$, $r, s, t \in \mathbb{Z}_q$
Output:	True if the argument is valid for the statement, false otherwise.
Suggested name:	`VerifyZArgument`

2.2.1 Pseudo-code

Require: $\overrightarrow{c_a} = (c_{a_1}, ..., c_{a_m}) \in \mathbb{G}_q^m$
Require: $\overrightarrow{c_b} = (c_{b_0}, ..., c_{b_{m-1}}) \in \mathbb{G}_q^m$
Require: $y \in \mathbb{Z}_q$
Require: $c_{a_0}, c_{b_m} \in \mathbb{G}_q$
Require: $\overrightarrow{c_D} = (c_{D_0}, ..., c_{D_{2m}}) \in \mathbb{G}_q^{2m+1}$
Require: $\vec{a}, \vec{b} \in \mathbb{Z}_q^m$
Require: $r, s, t \in \mathbb{Z}_q$
$elements \leftarrow []$ (start with an empty list)
for $i \in 1, ..., m$ **do**
  $elements$ += $publicCommitmentToString(c_{a_i}, p, q)$
end for
for $i \in 0, ..., m-1$ **do**
  $elements$ += $publicCommitmentToString(c_{b_i}, p, q)$
end for
$elements$ += $zeroProofInitialMessageToString(c_{a_0}, c_{b_m}, \overrightarrow{c_D}, p, q)$ {see paragraph 1.1.3.6}
$elements$ += $commitmentParamsToString(H, (G_1, ..., G_n), p, q)$ {see paragraph 1.1.3.5}
$elements$ += $zpGroupElementToString(pk, p, q)$ {see paragraph 1.1.3.2}
$input \leftarrow stringToBytes(reverseAndJoin(elements))$ {see paragraph 1.2.3}
$x \leftarrow RandomOracleHash(SHA256, input, 0, q)$
{Taking `RandomOracleHash` from alg 1.2.2}
if $c_{D_{m+1}} \neq 1$ **then**
  return false
end if
if $prodIncPow((c_{a_0}, c_{a_1}, ..., c_{a_m}), x, p, q) \neq computeCommitment(\vec{a}, r, ck)$ **then**
  return false
end if{Use the commitment function defined in alg 1.5}
if $prodIncPow((c_{b_m}, c_{b_{m-1}}, ..., c_{b_0})), x, p, q) \neq computeCommitment(\vec{b}, s, ck)$ **then**
  return false
end if{Note: the indices of $\overrightarrow{c_b}$ are reversed here}
{Note: the values $c_{a_1}, ..., c_{a_m}$ and $c_{b_0}, ..., c_{b_{m-1}}$ are taken from the statement, while the values c_{a_0} and c_{b_m} are taken from the argument.}
if $prodIncPow(c_D, x) \neq computeCommitment((bilinearMap(a, b, y)), t, ck)$ **then**
  return false
end if{Use the `bilinearMap` function defined in alg 1.7. The first parameter to the commitment computation is a vector with a single element, which is the result of the bilinearMap computation.}
{if no verification above has failed, the argument is convincing}
return true

Real-size values A valid argument is attached in the following file: .

2.3 Hadamard Product Argument Verification

Context:	Encryption Group: (p, q)
	Commitment key $ck = (H, G_1, ..., G_n)$
	Public key $pk \in \mathbb{G}_q$
	Number of rows m (needs to be ≥ 2)
	Number of columns n
Inputs:	Statement $(\overrightarrow{c_a}, c_b)$, where $\overrightarrow{c_a} = (c_{a_1}, ..., c_{a_m}) \in \mathbb{G}_q^m$, $c_b \in \mathbb{G}_q$
	Argument $(c_B, \pi_{\texttt{ZeroArg}})$, where $c_B = (c_{B_1}, ..., c_{B_n}) \in \mathbb{G}_q^m$, $\pi_{\texttt{ZeroArg}}$ is an argument as described in the inputs for section 2.2.
Output:	True if the argument is valid for the statement, false otherwise.
Suggested name:	`VerifyHArgument`

2.3.1 Pseudo-code

Require: $m \geq 2$
Require: $\overrightarrow{c_a} \in \mathbb{G}_q^m$
Require: $c_b \in \mathbb{G}_q$
Require: $\overrightarrow{c_B} \in \mathbb{G}_q^m$
Require: $c_{B_1} = c_{a_1}$
Require: $c_{B_m} = c_b$
$elements \leftarrow []$ (start with an empty list)
for $i \in 1, ..., m$ **do**
  $elements$ += $publicCommitmentToString(c_{a_i}, p, q)$
end for
$elements$ += $publicCommitmentToString(c_b, p, q)$
$elements$ += $hadamardProductProofInitialMessageToString(c_{a_0}, c_{b_m}, \overrightarrow{c_D}, p, q)$ {see paragraph 1.1.3.7}
$elements$ += $commitmentParamsToString(H, (G_1, ..., G_n), p, q)$ {see paragraph 1.1.3.5}
$elements$ += $zpGroupElementToString(pk, p, q)$ {see paragraph 1.1.3.2}
$firstInput \leftarrow stringToBytes(reverseAndJoin(elements))$ {see paragraph 1.2.3}
$x \leftarrow RandomOracleHash(SHA256, firstInput, 0, q)$
{Taking `RandomOracleHash` from alg 1.2.2}
$elements$ += `"1"` {see paragraph 1.1.3.2}
$secondInput \leftarrow stringToBytes(reverseAndJoin(elements))$ {see paragraph 1.2.3}
$y \leftarrow RandomOracleHash(SHA256, secondInput, 0, q)$
$c_{d_m} \leftarrow 1$
for $i \in [1, m-1]$ **do**

$exponent \leftarrow modExp(x, i, q)$
$c_{d_i} \leftarrow modExp(c_{B_i}, exponent, p)$
$c_{d_m} \leftarrow c_{d_m} * modExp(c_{B_{i+1}}, exponent, p) \bmod p$
end for
$\overrightarrow{c_d} \leftarrow (c_{d_1}, ..., c_{d_m})$
$\overrightarrow{-1} \leftarrow (-1, ..., -1)$ a vector of size n
$c_{-1} \leftarrow computeCommitment(\overrightarrow{-1}, 0, ck)$
$\overrightarrow{c_f} \leftarrow (c_{a_2}, ..., c_{a_m}, c_{-1})$
return $VerifyZArgument((\overrightarrow{c_f}, \overrightarrow{c_d}, y), \pi_{ZeroArg})$

Real-size test values A valid argument is attached in the following file: .

2.4 Product Argument Verification

Context:	Encryption Group: (p, q)
	Commitment key $ck = (H, G_1, ..., G_n)$
	Public key $pk \in \mathbb{G}_q$
	Number of rows m
	Number of columns n
Inputs:	Statement $(\overrightarrow{c_a}, b)$, where $\overrightarrow{c_a} = (c_{a_1}, ..., c_{a_m}) \in \mathbb{G}_q^m$, $b \in \mathbb{Z}_q$
	Argument $(c_b, \pi_{\texttt{HadamardArg}}, \pi_{\texttt{SingleVPA}})$, where $c_b \in \mathbb{G}_q$, $\pi_{\texttt{HadamardArg}}$ is an argument as described in the inputs for section 2.3, $\pi_{\texttt{SingleVPA}}$ is an argument as described in the inputs for section 2.1.
Output:	True if the argument is valid for the statement, false otherwise.
Suggested name:	`VerifyPArgument`

2.4.1 Pseudo-code

$minM \leftarrow max(m, 2)$ {The Hadamard argument only makes sense for $m \geq 2$.}
Require: $\overrightarrow{c_a} \in \mathbb{G}_q^{minM}$
Require: $b \in \mathbb{Z}_q$
Require: $\overrightarrow{c_b} \in \mathbb{G}_q$
$hadamardValid \leftarrow VerifyHArgument((\overrightarrow{c_a}, c_b), \pi_{\texttt{HadamardArg}})$ {The context for the VerifyHArgument call holds the value of $minM$, instead of m.}
$singleVpaValid \leftarrow VerifySVPArgument((c_b, b), \pi_{\texttt{SingleVPA}})$ {The context for the VerifySVPArgument call holds the original value of m.}
return $hadamardValid$ `AND` $singleVpaValid$

Real-size test values A simple valid argument is attached in the following file: . An argument with a value of $m = 1$ is attached in another file: . A more complex product argument is attached as: . This last one may take several minutes to verify.